Module 10 — Deploying AI Models with FastAPI (Docker & Minimal DevOps)

Learning Objectives

By the end of this module, you will be able to:

- Identify why traditional deployment methods fail for complex **AI dependencies** and how containers solve the "it works on my machine" problem.
- Distinguish between **development mode** (`fastapi dev`) and **production mode** (`fastapi run`).
- Construct a **production-ready Dockerfile** that utilizes caching to minimize build times for heavy ML environments.
- Manage **memory constraints** and CPU resources when serving resource-heavy machine learning models.
- Execute **graceful shutdowns** using the exec form of Docker commands.

Deployment Concepts for AI Engineers

1. Why Deployment is Hard for AI Systems

In machine learning, your application is not just your code; it is the specific version of Python, the exact versions of heavy libraries (like **PyTorch** or **Pydantic**), and the static model weights.

- **The Failure of "It Works on My Machine":** Differences in operating systems or installed system packages can cause a model to crash in production even if it runs perfectly in a local notebook.
- **Solution: Linux container images** (Docker) provide replicability and simplicity by packaging the entire environment into a single static unit.

2. ASGI Servers and Production Commands

For development, we use `fastapi dev`, which includes features like **auto-reload**. In production, we switch to **`fastapi run`**.

- **Uvicorn:** This is the high-performance server that loads and serves your application. `fastapi run` uses Uvicorn underneath to provide high-performance serving.
- **Environment Variables:** These are used to configure your application (like setting a `SECRET_KEY` or `DATABASE_URL`) without hardcoding them into the source code.

3. Docker for AI Engineers

- **What is a Container Image?** It is a **static** version of all the files, environment variables, and the default program (like your model API).
- **What is a Container?** It is the **running instance** of that image. It is essentially an isolated process running your model.
- **Why it matters for models:** Containers allow you to isolate your model's resource consumption. If a model uses 4GB of RAM, you can limit the container so it doesn't crash the entire host server.

Code Examples: The AI Inference Stack

1. Production-Ready FastAPI App (`app/main.py`)

This app is designed to be packaged and served in a professional environment.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class ModelInput(BaseModel):
    feature_vector: list[float]

@app.get("/")
def read_root():
    return {"status": "Model Server Live"}

@app.post("/predict")
def predict(data: ModelInput):
    # Simulated model logic
    prediction = sum(data.feature_vector) * 0.5
    return {"prediction": prediction}
```

Explanation: This setup uses standard Python type hints and Pydantic models for validation, ensuring that data arriving from the network is correctly converted to Python types before reaching the model logic.

2. Optimized Dockerfile for AI Inference

This Dockerfile uses a specific order of instructions to save time during development.

```
# (1) Start from the official Python base image
FROM python:3.9

# (2) Set the working directory inside the container
WORKDIR /code

# (3) Copy only requirements first to use Docker Cache
COPY ./requirements.txt /code/requirements.txt
```

```
# (4) Install dependencies (this part is slow, so we cache it)
RUN pip install --no-cache-dir --upgrade -r /code/requirements.txt

# (5) Copy the actual model code and weights
COPY ./app /code/app

# (6) Run the production server on port 80
CMD ["fastapi", "run", "app/main.py", "--port", "80"]
```

Line-by-Line Explanation:

- **`FROM python:3.9`**: Provides a stable, isolated Python environment.
- **`WORKDIR /code`**: Sets the location for all following commands.
- **`COPY ./requirements.txt ...`**: By copying this first, Docker can **cache** the installation step. If you change your code but not your requirements, Docker reuses the installed libraries instead of downloading them again.
- **`RUN pip install ...`**: Downloads the heavy AI libraries once.
- **`COPY ./app ...`**: Your code changes frequently, so this step is placed near the end to avoid breaking the cache for the libraries.
- **`CMD ["fastapi", "run", ...]`**: Uses the **exec form** (list of strings). This is critical because it allows the server to shut down gracefully and triggers **lifespan events**.

3. Running the Container Locally

To test the "production" environment on your own machine:

```
# Build the image and name it 'ai-service'
$ docker build -t ai-service .

# Run the container, mapping host port 80 to container port 80
$ docker run -d --name ai-container -p 80:80 ai-service
```

Explanation: The `-t` tags the image so you can find it later. The `-p 80:80` maps the network traffic so you can visit your API at `http://127.0.0.1`.

Real-World AI Deployment Scenarios

- **Deploying CPU-Based Models:** For simple models, you can run multiple **worker processes** inside a single container using the `--workers` flag to handle more traffic.
- **Updating Container Images Safely:** To update a model, you build a new image with a new tag. You start the new container, and only after its **startup events** are complete (model loaded) do you stop the old container.
- **Memory Management:** AI models are heavy consumers of memory. If you run multiple workers per container, you must ensure their combined memory usage doesn't exceed the machine's RAM.

- **Scaling Inference:** In a cluster, it is often better to run **one process per container** and replicate the entire container many times. This allows the system to distribute identical containers across different CPU cores or even different machines.

Common Deployment Mistakes AI Engineers Make

- **Using `fastapi dev` in Production:** Development mode is for coding; it is not optimized for high traffic or security.
- **Ignoring the Docker Cache:** Copying the entire directory *before* installing requirements. This forces Docker to re-download all libraries (like PyTorch) every time you fix a small typo in your code, wasting minutes of time.
- **Using the Shell Form for `CMD`:** Writing `CMD fastapi run ...` instead of `CMD ["fastapi", "run", ...]`. This prevents the container from stopping properly, which can cause issues during automated updates.
- **Hardcoding Ports:** Always setting the port to 8000 inside the code. In production containers, port 80 is the standard for web traffic.

Assignment: Dockerize an AI API

1. Create a folder `ai_app` with a `main.py` containing a POST `/predict` endpoint.
2. Create a `requirements.txt` containing `fastapi[standard]`.
3. Write a `Dockerfile` using the **optimized order** (Requirements -> Install -> Code).
4. Build the image using `docker build`.
5. Run the container and use the Swagger UI at `http://localhost/docs` to send a successful prediction.

Mini Project: Deployable Inference Service

Goal: Build a "Ready-to-Deploy" classification service.

- Create an API that uses a **Startup Event** to load a dummy model (Module 4).
- Use **Environment Variables** to set a "MODEL_VERSION" string that the API returns in every response.
- Create a Dockerfile that exposes port 80.
- Include a **Health Check** endpoint at `/health` that returns `{"status": "ok"}`.
- Verify that you can run the container locally and access it via your browser.

Production Readiness Checklist

- [ ] Is the app running using **`fastapi run`** (or Uvicorn with appropriate workers)?
- [ ] Does the `Dockerfile` use the **Exec Form** for the `CMD` instruction?
- [ ] Are **Requirements** copied and installed before the app code to leverage the Docker Cache?
- [ ] Has the **Memory usage** of the model been measured to set container limits?
- [ ] Is there an **Environment Variable** strategy for sensitive configurations?
- [ ] Does the API pass all tests when running inside the container?